

9장 추천 시스템

01 추천 시스템의 개요와 배경

추천 시스템

- 사용자 취향을 파악 → 맞춤형 콘텐츠, 상품 자동 추천하는 시스템
- 전자상거래, 콘텐츠 플랫폼에서 핵심 기술로 사용됨

중요한 이유

- 추천 엔진으로 수익 올릴 수 있음 ⇒ 업체들이 추천 엔진의 고도화에 큰 비용, 노력을 들이고 있음
- 사용자가 관심 가질 만한 콘텐츠 + 잊고 지냈거나 몰랐던 새로운 취향의 콘텐츠 추천 ⇒ 사용자가 오래 머무르게 됨
- 좋은 추천은 사용자의 사이트에 대한 신뢰도를 높임 ⇒ 재방문 유도
- 좋은 추천 시스템은 사용자가 무엇을 원하는 지 빠르게 찾아내 사용자의 온라인 쇼핑의 즐거움을 배가함

추천 시스템이 사용하는 데이터



유형

- Content based Filtering (초창기)
- Collaborative Filtering
 - Nearest Neighbor (초창기): 아마존
 - Latent Factor: 넷플릭스 추천 시스템 경연 대회에서 Matrix Facotrization 기법을 이용한 잠재 요인 협업 필터링 방식 우성 ⇒ 대부분이 이 방법 사용
- 개인화 특성 강화하기 위해 콘텐츠 기반 + 협업 기반 (하이브리드 형식)

02 콘텐츠 기반 필터링 추천 시스템

- 사용자가 특정한 아이템을 매우 선호 → 그 아이템과 비슷한 콘텐츠를 가진 다른 아이템 추천
- 예: 사용자가 특정 영화에 높은 평점 줌 → 그 영화의 장르, 배우, 감독, 영화 키워드 등의 콘텐츠와 유사한 다른 영화 추천

03 최근접 이웃 협업 필터링

협업 필터링 (Collaborative Filtering)

- 사용자가 아이템에 매긴 평점 정보, 상품 구매 이력같은 사용자 행동 양식만을 기반으로 추천을 수행하는 것
- 사용자가 평가한 다른 아이템을 기반으로 사용자가 평가하지 않은 아이템의 예측 평가를 도출함
- 친구들에게 물어보는 것과 유사한 방식
- 주요 목표: 사용자-아이템 매트릭스 같은 축적된 사용자 행동 데이터를 기반으로 사용자가 아직 평가하지 않은 아이템을 예측 평가하는 것
- Row: 개별 사용자 (사용자 아이디)
- Column: 개별 아이템 (아이템 아이디)

예시

- User1은 Item4에 대한 평점이 없음

사용자가 평가하지 않은 아이템을 평가한
아이템에 기반하여 예측 평가하는 알고리즘

	Item 1	Item 2	Item 3	Item 4
User 1	3		3	✓
User 2	4	2		3
User 3		1	2	2

- 판다스의 pivot_table() 함수 사용 → 사용자-아이템 평점 행렬 형태로 변경

로우 레벨 형태의 사용자 - 아이템 평점 데이터

UserID	Item ID	Rating
User 1	Item 1	3
User 1	Item 3	3
User 2	Item 1	4
User 2	Item 2	1
User 3	Item 4	5

변환

사용자 로우, 아이템 칼럼으로 구성된
사용자 - 아이템 평점 데이터

	Item 1	Item 2	Item 3	Item 4
User 1	3		3	
User 2	4	1		
User 3				5

사용자가 아이템에 대한 평점을 매기는 경우가 적음 ⇒ 희소 행렬 (Sparse Matrix) 특성을 갖고있음

최근접 이웃 협업 필터링

- 메모리 협업 필터링이라고 불림
- 종류
 - 사용자 기반(User-User): Customers like you also bought these items
 - 특정 사용자와 유사한 다른 사용자를 Top-N으로 선정 → 이 Top-N 사용자가 좋아하는 아이템 추천
 - 특정 사용자와 타 사용자 간의 유사도 측정 → 가장 유사도가 높은 Top-N 사용자 추출 → 그들이 선호하는 아이템 추천

		다크 나이트	인터스텔라	엣지오브 투모로우	프로메테우스	스타워즈 라스트제다이
상호간 유사도 높음	사용자 A	5	4	4		
	사용자 B	5	3	4	5	3
	사용자 C	4	3	3	2	5

사용자 A는 사용자 C보다 사용자 B와 영화 평점 측면에서 유사도가 높음. 따라서 사용자 A에게는 사용자 B가 재미있게 본 '프로메테우스'를 추천

- 아이템 기반(Item-Item): Customers who bought this item also bought these items

- 아이템이 가지는 속성과는 상관없이 사용자들이 그 아이템을 좋아하는지/싫어하는지의 평가 척도가 유사한 아이템을 추천하는 기준이 되는 알고리즘

		사용자 A	사용자 B	사용자 C	사용자 D	사용자 E
상호간 유사도 높음	다크 나이트	5	4	5	5	5
	프로메테우스	5	4	4		5
	스타워즈 라스트제다이	4	3	3		4

여러 사용자들의 평점을 기준으로 볼 때 '다크 나이트'와 가장 유사한 영화는 '프로메테우스'

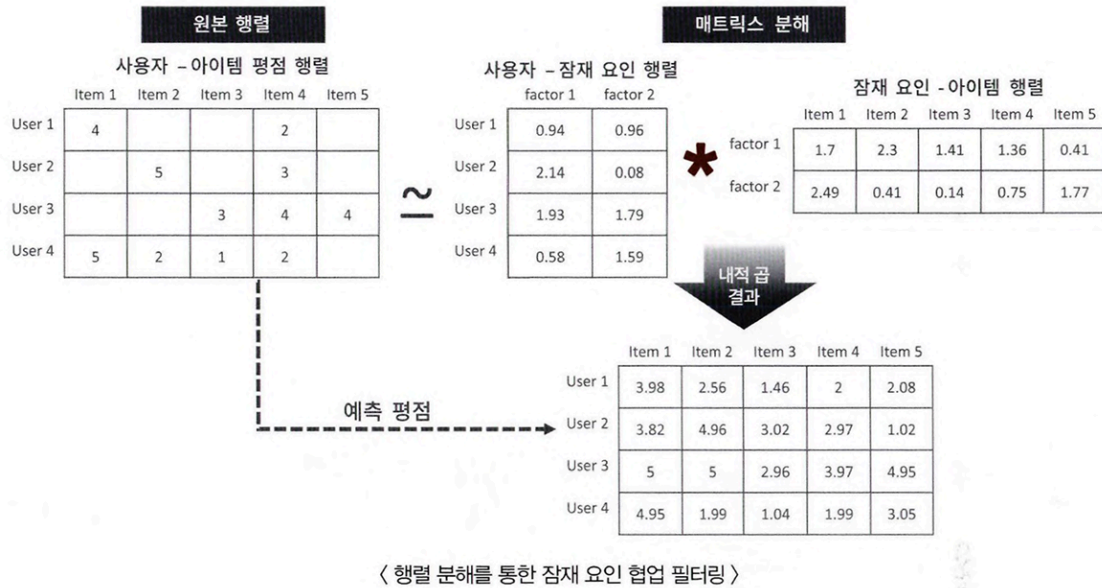
- 일반적으로, 아이템 기반 협업 필터링이 정확도가 더 높음
- 추천 시스템에 사용되는 데이터는 피처 벡터화된 텍스트 데이터와 동일하게 **다차원 희소 행렬** ⇒ 코사인 유사도 이용

04 잠재 요인 협업 필터링

정의

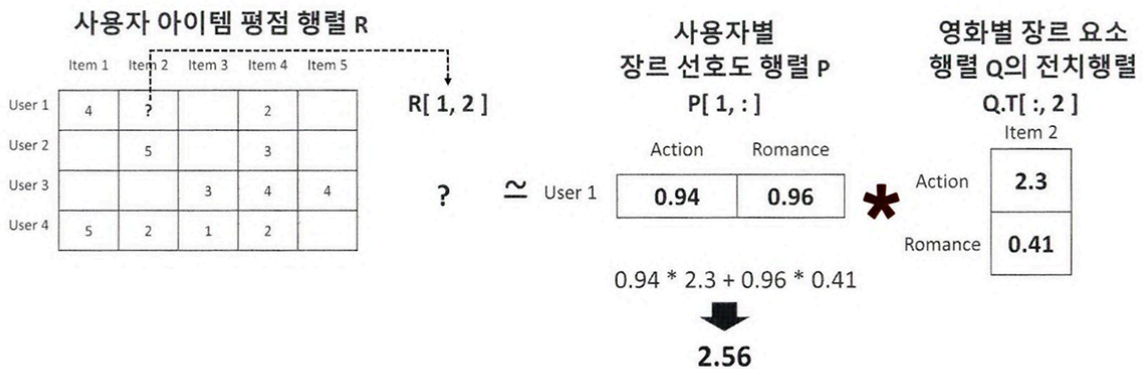
- 사용자-아이템 평점 매트릭스 속에 숨어 있는 잠재 요인을 추출해 추천 예측 할 수 있게 하는 기법
- 행렬 분해: 대규모 다차원 행렬을 SVD와 같은 차원 감소 기법으로 분해하는 과정에서 잠재 요인 추출

행렬 분해를 통한 잠재 요인 협업 필터링



1. 잠재 요인을 기반으로 사용자-아이템 행렬 데이터를 저차원 밀집 행렬의 사용자-잠재 요인 행렬과 아이템-잠재 요인 행렬의 전치행렬로 분해
2. 분해된 두 행렬의 내적 → 새로운 예측 사용자-아이템 평점 행렬 데이터 만들
3. 사용자가 아직 평점을 부여하지 않은 아이템에 대한 예측 평점 생성

예시-영화 추천

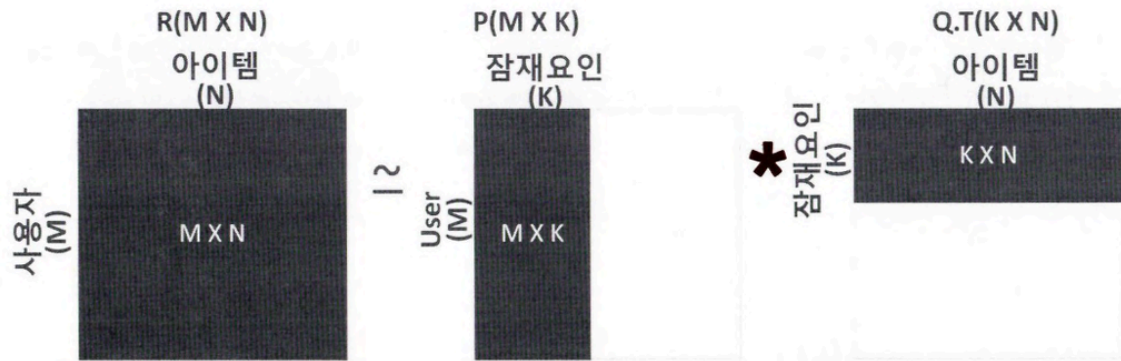


- User1: 액션, 로맨스 장르에 대한 선호도 높음 (P행렬)
- Item1: 액션 요소가 강한 영화 (Q행렬)
- 내적 계산을 통해 User1이 Item1에 줄 평점 예측 가능

행렬 분해

- 다차원의 매트릭스를 저차원 매트릭스로 분해하는 기법
- 대표 기법: SVD, NMF

핵심 아이디어



- R: 원본 사용자-아이템 평점 행렬, 크기: $M \times N$
- P: 사용자-잠재 요인 행렬, 크기: $M \times K$
- Q: 아이템-잠재 요인 행렬, 크기: $N \times K$
- K: 잠재 요인의 수 (예: 액션 선호도, 로맨스 선호도)
- M: 총 사용자 수
- N: 총 아이템 수
- $R = P * Q.T$: $M \times N$ 차원의 사용자-아이템 평점 행렬
- Q.T: Q 매트릭스의 행과 열 값을 교환한 전치 행렬

예시-NaN값을 많이 가지는 고차원의 희소 행렬 R 분해 → 밀집 행렬 P, Q로 분해

R (4 X 5)					
	Item 1	Item 2	Item 3	Item 4	Item 5
User 1	4			2	
User 2		5		3	
User 3			3	4	4
User 4	5	2	1	2	

P (4 X 2)		
	factor 1	factor 2
User 1	0.94	0.96
User 2	2.14	0.08
User 3	1.93	1.79
User 4	0.58	1.59

Q.T (2 X 4)					
	Item 1	Item 2	Item 3	Item 4	Item 5
factor 1	1.7	2.3	1.41	1.36	0.41
factor 2	2.49	0.41	0.14	0.75	1.77

개별 Row는 개별 User에 대한 Latent Factor들을 반영
개별 Column은 개별 item에 대한 Latent Factor들을 반영

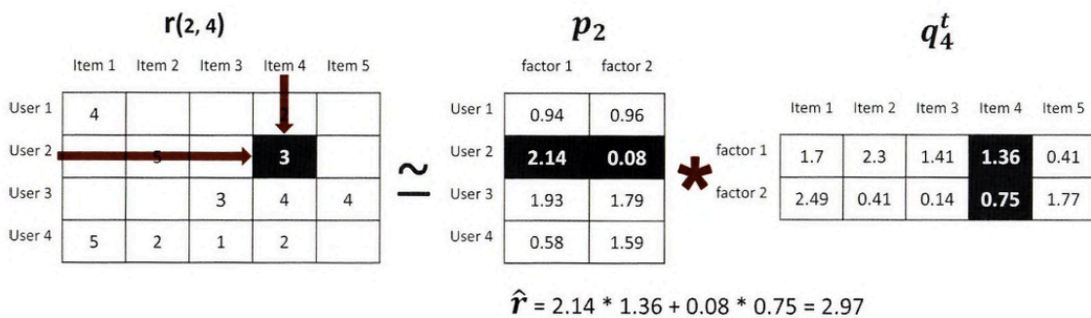
$r(u, i)$: R행렬의 u행 사용자와 i열 아이템 위치에 있는 평점 데이터

$$r_{u,i} = p_u * q_i^t$$

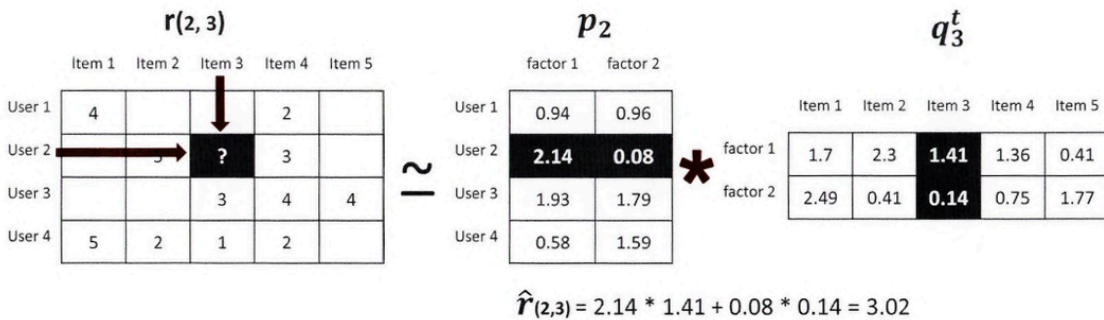
p_u : P 행렬에서 u행 사용자의 벡터

q_i^t : Q행렬의 u행 아이템 벡터의 전치 벡터

$r(2,4)$ 의 예측값 계산



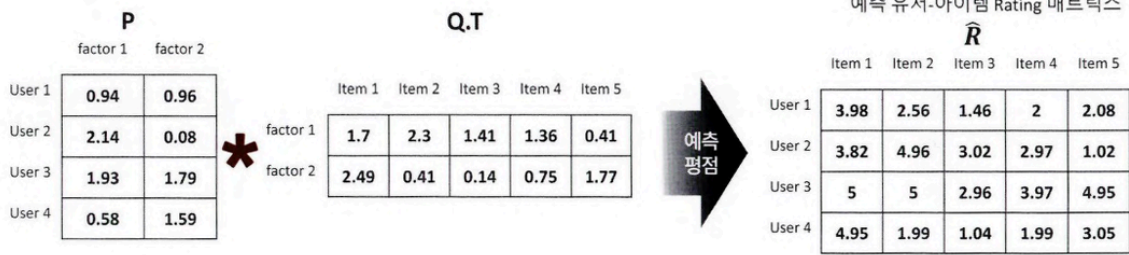
사용자가 평가하지 않은 아이템에 대한 평점도 잠재 요인으로 분해된 P, Q 행렬로 예측 가능



$r(2,3)$ 은 사용자가 평점을 매기지 않은 NaN이지만, $r(2,3)=p_2*q_3^t$ 으로 유추 가능

사용자-아이템 평점 행렬의 미정 값을 포함한 모든 평점 값은 행렬 분해를 통해 얻어진 P행렬과 Q.T 행렬의 내적을 통해 예측 평점으로 다시 계산 가능

$$R \cong \hat{R} = P * Q.T$$



행렬 분해 방식

- SVD: NaN값 없는 행렬에만 적용 가능.. R 행렬 적용 불가능 $\Rightarrow$ SGD, ALS 방식을 이용해 SVD 수행
- SGD(확률적 경사 하강법)
- ALS

확률적 경사 하강법을 이용한 행렬 분해

- P, Q 행렬로 계산된 예측 R 행렬 값이 실제 R 행렬 값과 가장 최소의 오류를 가질 수 있도록 반복적인 비용 함수 최적화를 통해 P, Q 유추
- L2 규제를 반영해 실제 R 행렬 값과 예측 R 행렬 값의 차이를 최소화하는 방향성을 가지고 P행렬과 Q 행렬에 업데이트 값을 반복적으로 수행하면서 최적화된 예측 R 행렬 구하기

절차

1. P, Q를 임의의 값을 가진 행렬로 설정
2. P와 Q.T 값을 곱해 예측 R 행렬 계산하고 예측 R 행렬과 실제 R 행렬에 해당하는 오류 값 계산
3. 이 오류값을 최소화할 수 있도록 P, Q 행렬을 적절한 값으로 각각 업데이트
4. 만족할 만한 오류 값을 가질 때까지 2, 3 번 반복하면서 P, Q를 업데이트해 근사화

<비용 함수식>

$$\min \sum (r_{u,i} - p_u q_i^t)^2 + \lambda (\|q_i\|^2 + \|p_u\|^2)$$

<비용 함수 업데이트 식>

$$\begin{aligned} \dot{p}_u &= p_u + \eta (e_{(u,i)} * q_i - \lambda * p_u) \\ \dot{q}_i &= q_i + \eta (e_{(u,i)} * p_u - \lambda * q_i) \end{aligned}$$

- p_u : P 행렬의 사용자 u행 벡터
- q_i^t : Q 행렬의 아이템 i행의 전치 벡터(transpose vector)
- $r_{u,i}$: 실제 R 행렬의 u행, i열에 위치한 값.
- $\hat{r}_{u,i}$: 예측 R 행렬의 u행, i열에 위치한 값. $p_u * q_i^t$ 로 계산
- $e_{(u,i)}$: u행, i열에 위치한 실제 행렬 값과 예측 행렬 값의 차이 오류. $r_{u,i} - \hat{r}_{u,i}$ 로 계산
- η : SGD 학습률
- λ : L2 규제(Regularization) 계수

08 파이썬 추천 시스템 패키지 -Surprise

- 추천 시스템 전용 파이썬 라이브러리
- 다양한 협업 필터링 알고리즘을 간편하게 사용할 수 있도록 설계됨
- 사이킷런과 유사한 API구조 제공 → 사용 쉬움
- 지원하는 기능: 추천 시스템 구현, 모델 학습, 예측, 성능 평가, 하이퍼 파라미터 튜닝

Surprise 특징

- 다양한 알고리즘 제공: SVD, SVD++, NMF, KNNBasic, KNNBaseline, CoClustering 등
- 간단한 API: fit(), predict(), train_test_split(), cross_validate(), GridSearchCV 클래스

Surprise 주요 모듈

API 명	내용
Dataset.load_builtin (name='ml-100k')	무비렌즈 아카이브 FTP 서버에서 무비렌즈 데이터를 내려받습니다. ml-100k, ml-1M를 내려받을 수 있습니다. 일단 내려받은 데이터는 .surprise_data 디렉터리 밑에 저장되고, 해당 디렉터리에 데이터가 있으면 FTP에서 내려받지 않고 해당 데이터를 이용합니다. 입력 파라미터인 name으로 대상 데이터가 ml-100k인지 ml-1m인지를 입력합니다(name='ml-100k'). 디폴트는 ml-100k입니다.

API 명	내용
Dataset.load_from_file (file_path, reader)	OS 파일에서 데이터를 로딩할 때 사용합니다. coma, 탭 등으로 칼럼이 분리된 포맷의 OS 파일에서 데이터를 로딩합니다. 입력 파라미터로 OS 파일명, Reader로 파일의 포맷을 지정합니다.
Dataset.load_from_df (df, reader)	판다스의 DataFrame에서 데이터를 로딩합니다. 파라미터로 DataFrame을 입력받으며 DataFrame 역시 반드시 3개의 칼럼인 사용자 아이디, 아이템 아이디, 평점 순으로 칼럼 순서가 정해져 있어야 합니다. 입력 파라미터로 DataFrame 객체, Reader로 파일의 포맷을 지정합니다.

line_format (string) : 칼럼을 순서대로 나열함. 입력된 문자열을 공백으로 분리해 칼럼으로 인식함

sep (char) : 칼럼을 분리하는 분리자. default: `\t`. 판다스 DataFrame에서 입력받을 경우에는 기재할 필요 없음

rating_scale (tuple, optional) : 평점 값의 최소 ~ 최대 평점 설정. default: (1, 5)

Surprise 추천 알고리즘 클래스

- SVD: 행렬 분해를 통한 잠재 요인 협업 필터링을 위한 SVD 알고리즘
 - 주요 파라미터

파라미터명	내용
n_factors	잠재 요인 K의 개수. 디폴트는 100. 커질수록 정확도가 높아질 수 있으나 과적합 문제가 발생할 수 있습니다.
n_epochs	SGD(Stochastic Gradient Descent) 수행 시 반복 횟수, 디폴트는 20.
biased (bool)	베이스라인 사용자 편향 적용 여부이며, 디폴트는 True입니다.

- KNNBasic: 최근접 이웃 협업 필터링을 위한 KNN 알고리즘
- BaselineOnly: 사용자 Bias와 아이템 Bias를 감안한 SGD 베이스라인 알고리즘

알고리즘 유형	RMSE	MAE	Time
SVD	0.934	0.737	0:00:11
SVD++	0.92	0.722	0:09:03
NMF	0.963	0.758	0:00:15
Slope One	0.946	0.743	0:00:08
k-NN	0.98	0.774	0:00:10
Centered k-NN	0.951	0.749	0:00:10
k-NN Baseline	0.931	0.733	0:00:12
Co-Clustering	0.963	0.753	0:00:03
Baseline	0.944	0.748	0:00:01

SVD++ 알고리즘의 RMSE, MAE 성적이 가장 좋지만, 너무 오래걸려서 쓰기 어려움
k-NN 자체는 상대적으로 낮지만, Baseline 결합하면 높아짐

베이스라인 평점

- Baseline: 각 개인이 평점을 부여하는 성향을 반영해 평점을 계산하는 방식
- Baseline Rating: 개인의 성향을 반영해 아이템 평가에 bias 요소를 반영해 평점을 부과하는 것
- 공식: 전체 평균 평점 + 사용자 편향 점수 + 아이템 편향 점수
 - 전체 평균 평점 = 모든 사용자의 아이템에 대한 평점을 평균한 값
 - 사용자 편향 점수 = 사용자별 아이템 평점 평균 값 - 전체 평균 평점
 - 아이템 편향 점수 = 아이템별 평점 평균 값 - 전체 평균 평점



모든 사용자의 평균 영화 평점 : **3.5**



난 진정한 영화 매니아.
영화 평가는 언제나 깐깐하게

사용자 A 평균 평점
3.0

사용자 A의 어벤저스 3편 베이스 라인 평점 = $3.5 - 0.5 + 0.7 = 3.7$

모든 사용자의 평균 영화 평점	+	사용자 편향 점수	+	아이템 편향 점수
3.5		$3.0 - 3.5 = -0.5$		$4.2 - 3.5 = 0.7$

어벤저스 3편 평균 평점

4.2

